

Introduction

Project Background and Purpose

The Café Fausse Restaurant Platform was implemented as a full-stack restaurant website that supports restaurant information, menu browsing, table reservations, newsletter sign-up, customer engagement, and Click & Collect ordering. Customers can use the public interface to view menu items, make reservations, add order items to a cart, complete checkout details, and receive an order confirmation. The system also includes a protected staff route for reviewing order information, while wider management functions such as full menu editing and customer-account administration remain future enhancements.

The platform strengthens the customer experience by providing clear navigation, responsive pages, database-backed menu content, a reservation form, a newsletter subscription option, and social-media links. It also reduces some manual processing by storing reservations, newsletter subscriptions, and order details in the database. The final system therefore provides a practical foundation for future development while keeping the implemented scope realistic and aligned with the working application.

Scope

a. Core Requirements

The Café Fausse Website is a responsive full-stack web application that presents restaurant information, displays database-backed menu content, supports table reservations, showcases gallery content and customer testimonials, and captures newsletter subscriptions. The project uses React with JSX for the front end, Flask for the back end, and PostgreSQL for persistent data management. **(Quantic 2026)**

b. Enhancements

The project extends the SRS through selected enhancements that create a more realistic and user-centred restaurant platform. These enhancements include data-driven featured specials, online Click & Collect ordering, cart and checkout functionality, pay-on-collection confirmation, persistent order records, a configurable external Uber Eats delivery hand-off, improved navigation, responsive design, social-media links, and protected staff order access.

These additions complement the original specification and support a more comprehensive solution without changing the core requirements.

Requirements Analysis & Solution Design

Product Goals

The following measurable goals define the expected outcomes of the Café Fausse Restaurant Platform. They are derived from the Software Requirements Specification (**Quantic 2026**) and extended where appropriate to support the approved architectural enhancements.

Customer Goals		
ID	Goal	Success Measure
CG-01	Allow customers to book tables through the website without needing direct assistance	Customers are able to complete reservations without requiring staff support
CG-02	Give customers clear, reliable access to current menu details	Menu content displays correctly and remains accessible on supported devices
CG-03	Enable customers to sign up for restaurant newsletter updates	Newsletter sign-up details are captured successfully by the system
CG-04	Create a user-friendly customer journey supported by responsive pages and optional ordering features	Acceptance testing confirms that users can navigate and use the platform effectively
Business Goals		
BG-01	Strengthen customer engagement through a useful and informative web presence	Website activity and newsletter sign-ups show measurable growth
BG-02	Minimise the amount of manual work involved in handling reservations	Staff spend less time processing reservation requests manually
BG-03	Broaden the restaurant's customer services by adding optional online ordering	Online ordering is implemented as an added service feature
Technical Goals		
TG-01	Build the system using a modular full-stack structure based on React, Flask, and PostgreSQL	Main system components are completed in a way that supports ongoing maintenance
TG-02	Design the architecture so approved future features can be added as the platform develops	New features can be introduced without major changes to the existing architecture

Epics

The functional requirements are organised into Agile epics that group related business capabilities. While the product goals define measurable outcomes, the epics organise related functions into delivery areas.

ID	Epic Name	Description	Related Requirements Area	Primary Stakeholders
EP-01	Customer Account Management	Support the creation, secure access, and ongoing updating of customer accounts.	User Accounts, Authentication	• Customers • Administrators
EP-02	Menu Management	Present menu items clearly and allow restaurant staff to keep menu details up to date.	Menu Viewing, Menu Administration	• Customers • Restaurant Management
EP-03	Reservation Management	Provide tools for customers and staff to make, update, and oversee table bookings.	Reservations	• Customers • Restaurant Staff • Managers
EP-04	Takeaway Ordering	Allow customers to place takeaway orders through the online platform.	Ordering	• Customers • Restaurant Staff • Managers
EP-05	Customer Communication	Handle customer messages, booking confirmations, and service notifications in one place.	Contact, Notifications	• Customers • Restaurant Management
EP-06	Restaurant Information	Share key restaurant information such as location, trading hours, and contact details.	Information Pages	• Customers • Restaurant Management
EP-07	Newsletter Management	Maintain newsletter sign-ups and support ongoing promotional communication.	Marketing	• Restaurant Management

Website Functional Areas Implemented

The platform's functional scope was derived from the project Software Requirements Specification (**Quantic 2026**). The implemented functional areas include restaurant information pages, menu browsing, table reservations, gallery content, customer testimonials, newsletter subscription functionality, online ordering, checkout, order confirmation, a social-media hub, and

protected staff order review. Additional features were incorporated to create a more practical restaurant platform while preserving alignment with the original project requirements.

Identified Opportunities

The supplied SRS establishes a focused baseline for a modern restaurant website by defining the core features required for the assessment (**Quantic 2026**). The design review identified opportunities to improve customer interaction, operational support, and future scalability. These opportunities extend the solution beyond a basic informational website through implemented features such as online ordering, Click & Collect, database-backed menu data, customer reservations, order confirmation, newsletter capture, reliable gallery assets, and a more refined responsive user experience. The enhancements support the original requirements without changing the functionality specified in the SRS (**Quantic 2026**).

Engineering Enhancements

The engineering enhancements demonstrate how the solution was extended beyond the minimum SRS scope through improvements to architecture, data modelling, user workflows, and interface design (**Quantic 2026**). The main additions include data-driven specials, Click & Collect ordering, cart quantity changes, checkout summary, pay-on-collection status, persistent order records, a configurable Uber Eats delivery hand-off, protected staff order access, local gallery assets, and responsive interface refinements. Together, these enhancements reposition the website as a broader digital restaurant platform while retaining compliance with the original specification (**Quantic 2026**).

System Design

Overall Architecture

The Café Fausse Restaurant Platform uses a three-layer architecture comprising a React-based client interface, a Flask application layer, and a PostgreSQL relational database. This approach satisfies the technology requirements outlined in the Software Requirements Specification while supporting separation of concerns, maintainability, and future scalability (**Quantic 2026**).

React delivers the responsive client interface, Flask manages application logic and API communication, and PostgreSQL stores persistent data such as menu items, reservations, orders, order details, newsletter subscriptions, gallery content, testimonials, and staff user data. Customer contact details are stored with the reservation or order that requires them rather than through a separate customer-account feature. This architecture also accommodates added features such as online ordering and Click & Collect without disrupting the requirements defined in the SRS (**Quantic 2026**).

The layered approach separates presentation, application, and data responsibilities, which makes the platform easier to maintain and extend over time. The backend also uses migrations and seeded data so that menu items, testimonials, gallery records, and staff access can be managed consistently during development.

Technology Stack

- The technology stack supports the project specification and a maintainable full-stack workflow **(Quantic 2026)**.
- React supports the client-side interface through reusable components and responsive user experience design **(Meta 2026)**.
- Flask provides a lightweight server-side framework for web services and API-driven application logic **(Pallets Projects 2026)**.
- PostgreSQL provides structured, persistent, and reliable relational data storage **(PostgreSQL Global Development Group 2026)**.
- Visual Studio Code served as the primary development environment, while GitHub managed version control and source code history **(Microsoft 2026; GitHub 2026a)**.
- GitHub Copilot assisted implementation through AI-supported code suggestions, while ChatGPT supported requirements analysis, architecture planning, documentation, and implementation guidance **(GitHub 2026b; OpenAI 2026a)**.
- AI image generation tools created original visual assets for the website's brand identity and interface design **(OpenAI 2026b)**.

Information Architecture

The information architecture provides a clear navigation structure that helps users find key content and complete common tasks efficiently. The customer-facing navigation includes Home, About Us, Menu, Reservations, Order Online, Gallery, and Follow Us. Transactional pages such as Checkout and Order Confirmation are reached through the ordering journey rather than being presented as main navigation items. The previous contact-focused route is represented through social-media links, while staff login and the protected order dashboard remain outside the public navigation. This structure supports usability, limits unnecessary navigation complexity, and allows the platform to be expanded in future with minimal structural changes.

User Journeys

The user journeys define the main ways in which customers interact with the Café Fausse platform and ensure that frequent tasks can be completed clearly and efficiently. The primary journeys include viewing restaurant information, browsing menu items, completing a table reservation, signing up for the newsletter, and following the restaurant through configured social links. The enhanced ordering journey allows a customer to move from a featured special or the

Order Online page to the cart, adjust quantities, review the checkout summary, enter contact and collection details, confirm pay-on-collection, and receive an order confirmation. A separate external delivery journey sends users to the configured Uber Eats destination, while staff access is handled through a direct protected login route.

Database Design

The relational database structure preserves data consistency, reduces unnecessary duplication, and supports reliable data management. The operational Flask and PostgreSQL schema includes menu items, reservations, orders, order items, newsletter subscribers, gallery images, testimonials, and admin users. Order items link customer orders to menu items, while order records store customer contact details, collection information, totals, and payment status. The implemented schema does not include a separate customer-account table; customer details are captured only where they are needed for reservations and orders. This keeps the final database aligned with the working application while still supporting the requirements defined in the SRS **(Quantic 2026)**.

This design provides a scalable data foundation that can support future features such as customer accounts, online card payments, fuller menu administration, and reservation management without requiring major changes to the existing platform structure.

User Interface Design

The user interface provides a contemporary and responsive experience aligned with the premium identity of the Café Fausse brand. The final design uses a black, ivory, and gold visual palette, refined heading typography, readable body text, consistent buttons, responsive card layouts, and clear active navigation states to create visual continuity across the application. The interface prioritises clear content presentation, simple navigation, and adaptable layouts for desktop, tablet, and mobile use.

Locally bundled restaurant imagery supports a reliable gallery and professional visual presentation without relying on remote image sources. The checkout page follows a familiar two-part pattern by showing customer and collection details alongside an order summary with quantities and totals. Overall, the completed interface combines visual design and usability principles to support efficient task completion and a clearer customer experience.

AI-Assisted Software Engineering Approach

AI Development Methodology

Generative AI served as a support mechanism across the project's research, design, development, testing, refactoring, and documentation activities. ChatGPT supported requirements interpretation, solution design, system architecture planning, database modelling, implementation preparation, debugging, route review, test planning, and technical documentation drafting (**OpenAI 2026a**). It also helped refine design decisions, clarify technical topics, assess implementation outputs, and improve the written report (**OpenAI 2026c**).

During development, GitHub Copilot supported code completion, code generation, and implementation work for the React frontend, Flask backend, and PostgreSQL database (**GitHub 2026b**). AI-assisted review was also used to resolve issues such as API and CORS communication, migration and seed alignment, stale cart behaviour, mismatched specials and orderable items, gallery image reliability, and the need to keep staff routes protected from public navigation (**OpenAI 2026c**). AI image generation tools created original visual assets for the Café Fausse brand and user interface, while GitHub managed version control and source code history (**OpenAI 2026b; GitHub 2026a**).

All AI-assisted outputs, including research guidance, architecture suggestions, documentation, images, and source code, were reviewed and refined before inclusion in the project. This process kept the final work consistent with the Software Requirements Specification and assignment expectations. As a result, AI supported the development process while independent judgement, responsibility, and academic accountability for the final solution were maintained.

Tool	Purpose
ChatGPT	Requirements engineering, architecture, documentation, prompt engineering and code review
GitHub Copilot	AI-assisted code generation and implementation
GitHub	Version control and source code management
Visual Studio Code	Integrated development environment (IDE)
React	Frontend development framework
Flask	Backend web framework
PostgreSQL	Relational database management system
AI Image Generator	Creation of restaurant branding and visual assets

Conclusion

This report presented the requirements analysis and design approach for the Café Fausse Restaurant Platform, translating the Software Requirements Specification into a completed responsive full-stack web application that supports restaurant information, menu browsing, table reservations, newsletter subscription, gallery content, customer testimonials, social-media

engagement, and selected enhancements such as Click & Collect ordering, checkout, order confirmation, persistent order records, an external Uber Eats delivery hand-off, protected staff order access, local visual assets, and a modular architecture (**Quantic 2026**).

The React, Flask, and PostgreSQL design separates presentation, application logic, and data management responsibilities, supporting maintainability, scalability, and clear system organisation. AI-assisted tools were used as support during research, design, planning, documentation, and development, with outputs reviewed and refined before inclusion to maintain academic accountability, technical justification, and alignment with the project scope (**OpenAI 2026a; GitHub 2026b**).

References

1. **Quantic. 2026.** *MSAIEE Web Application and Interface Design: Café Fausse Software Requirements Specification*. Accessed July 24, 2026.
https://uploads.smart.ly/emails/Projects/MSSE/MSEE_Web_Application_and_Interface_Design_Cafe_Fausse_SRS.pdf
2. **Meta. 2026.** “React: The Library for Web and Native User Interfaces.” Accessed July 26, 2026. <https://react.dev/>.
3. **Pallets Projects. 2026.** “Welcome to Flask.” *Flask Documentation*. Accessed July 26, 2026. <https://flask.palletsprojects.com/>.
4. **PostgreSQL Global Development Group. 2026.** “PostgreSQL Documentation.” Accessed July 26, 2026. <https://www.postgresql.org/docs/>.
5. **Vite. 2026.** “Vite: Next Generation Frontend Tooling.” Accessed July 26, 2026. <https://vite.dev/>.
6. **Microsoft. 2026.** “Documentation for Visual Studio Code.” Accessed July 26, 2026. <https://code.visualstudio.com/Docs>.
7. **GitHub. 2026a.** “GitHub.” Accessed July 26, 2026. <https://github.com/>.
8. **GitHub. 2026b.** “GitHub Copilot Documentation.” *GitHub Docs*. Accessed July 26, 2026. <https://docs.github.com/en/copilot>.
9. **OpenAI. 2026a.** ChatGPT response to project prompts on requirements analysis, architecture planning, documentation, and implementation guidance. Generated through ChatGPT, July 2026. <https://chatgpt.com/>.
10. **OpenAI. 2026b.** AI image-generation responses used to create original Café Fausse visual assets. Generated through OpenAI tools, July 2026. <https://openai.com/>.
11. **OpenAI. 2026c.** ChatGPT response to project-report comparison and revision prompts for aligning the original report with the generated implementation report. Generated through ChatGPT, July 26, 2026. <https://chatgpt.com/>.

AI-Assisted Development Conversation Log

AI provided structured support across the project's research, analysis, design, planning, documentation, and implementation activities.

Separate AI conversations supported specific SDLC phases, improving focus, context management, and relevance. These conversations produced artefacts such as requirements analysis, UX research, architecture, database design, testing strategy, and technical documentation.

All AI outputs were reviewed, refined, and checked against the SRS, assignment objectives, and software engineering best practices before inclusion. Overall, the AI conversations supported project decision-making while final responsibility for the design and implementation remained with me. The table below records the main conversations, their purposes, and links for transparency and reproducibility.

Chat Purpose	Link
00 Prompt Engineering	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a6113d7-d344-83ea-80e1-5e4af163a150
01 Requirements Engineering	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a61214c-528c-83ea-9925-d47f874045fc
02 UX Research	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a6127b5-37e8-83ea-bf33-582c527ea7a8
03 UI/UX Design	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a612814-e464-83ea-a71b-24b733d04178
04 Information Architecture	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a612865-f764-83ea-9490-9dbb16ff2ca1
05 Database Design	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a612ba9-8658-83ea-95c2-734e89167b73
06 Backend Architecture	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a612c25-7150-83ea-96e9-00dd45afdddc
07 Frontend Architecture	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a612c6d-2fbc-83ea-aba5-8f58a72e3352

08 AI Image Strategy	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a612d56-f284-83ea-86de-0142fcfc836c
09 AI Development Strategy	https://chatgpt.com/g/g-p-6a611417cd048191ac6a5e7b3b02ec9d/c/6a612d95-e6f4-83ea-95d7-27015e01a281